Week 3 • Class 2 — Embeddings & Semantic Search

Embeddings • Semantic Search • Vector DB
Keyword Retrieval → Meaning Retrieval

Continuity: Week 1 → Week 3

- Class 1 → Retrieval architecture (context + Top-K + chunking)
- Class 2 → Retrieval intelligence (embeddings + semantic similarity)
- Same model. Smarter retrieval layer.
- Goal: “return policy” should match “refund rules”.

Why Keyword Search Breaks

- Keyword search matches words, not meaning
- Query: “refund policy” → likely match
- Query: “return rules” → may fail (different words, same meaning)
- We need semantic similarity.

Different words
Same meaning
→ Embeddings fix retrieval

Embeddings: Meaning → Vector

- Embedding = vector representation of meaning
- Text → list[float] (hundreds/thousands of dimensions)
- Similar meaning → vectors are close in space
- Retrieval becomes ranking by similarity score

Semantic Retrieval Pipeline

System Flow (Meaning Retrieval)

Search becomes math: vector similarity

Code Demo

Create Embedding Vector

```python
from openai import OpenAI
client = OpenAI()

def get_embedding(text: str) -> list[float]:
    r = client.embeddings.create(
        model="text-embedding-3-small",
        input=text
    )
    return r.data[0].embedding

vec = get_embedding("refund policy")
print("dims:", len(vec))
```

Code Demo

Cosine Similarity (Ranking)

```python
import numpy as np

def cosine_similarity(a, b):
    a = np.array(a, dtype=float)
    b = np.array(b, dtype=float)
    return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)))

v1 = get_embedding("refund policy")
v2 = get_embedding("return rules")
print("similarity:", cosine_similarity(v1, v2))
```

Code Demo

Precompute Document Embeddings

```
documents = [
  "Refunds are allowed within 30 days with receipt.",
  "Digital products are not refundable.",
  "Damaged items can be replaced or issued store credit.",
  "Shipping usually takes 5-7 business days."
]

doc_vectors = [(d, get_embedding(d)) for d in documents]
```

Code Demo

Semantic Search = Rank + Top-K

```python
def semantic_search(query: str, k: int = 2):
    qv = get_embedding(query)
    scored = []
    for doc, dv in doc_vectors:
        s = cosine_similarity(qv, dv)
        scored.append((doc, s))
    scored.sort(key=lambda x: x[1], reverse=True)
    return scored[:k]

print(semantic_search("return policy", k=2))
```

Code Demo

Semantic RAG Integration

```python
def rag_semantic(question: str, k: int = 2) -> str:
    topk = semantic_search(question, k=k)
    context = "\n".join([doc for doc, _ in topk])

    prompt = f"""Use ONLY the context to answer.
Context:
{context}

Question: {question}
If not in context, say "I don't know"."""

    resp = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role":"user","content": prompt}],
        temperature=0.0
    )
    return resp.choices[0].message.content

print(rag_semantic("Can I return after 2 weeks?", k=2))
```

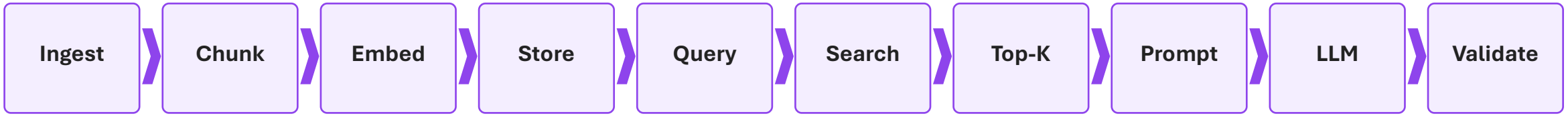
We didn’t upgrade the model.
We upgraded retrieval intelligence.

Vector DB

- List-of-vectors does not scale cleanly to 10K → 1M+ docs
- Vector DB stores embeddings and supports fast similarity search
- Capabilities: indexing • persistence • metadata filters • Top-K
- Examples: FAISS/Chroma (local), Pinecone/Weaviate (hosted)

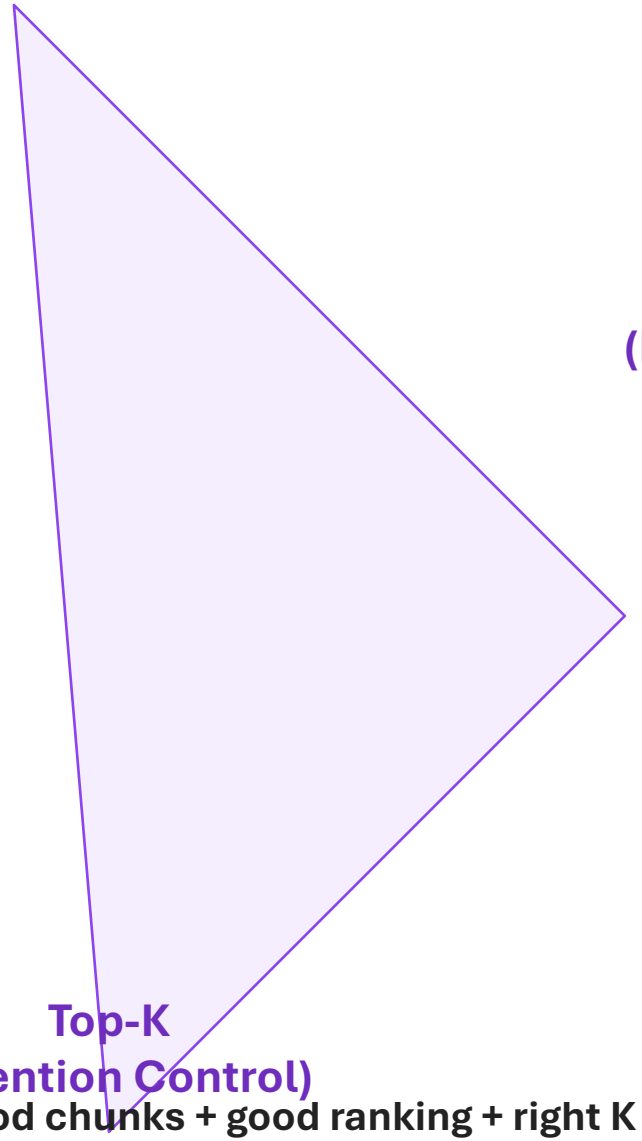
Production View (End-to-End)

RAG System Architecture



Elite Insight

RAG Quality Triangle (Elite Insight)



Good RAG quality = good chunks + good ranking + right K

Top-K Tradeoff

Top-K Tradeoff (Signal vs Noise)

K = 1

Too small
may miss info

K = 3

Good default
focused + sufficient

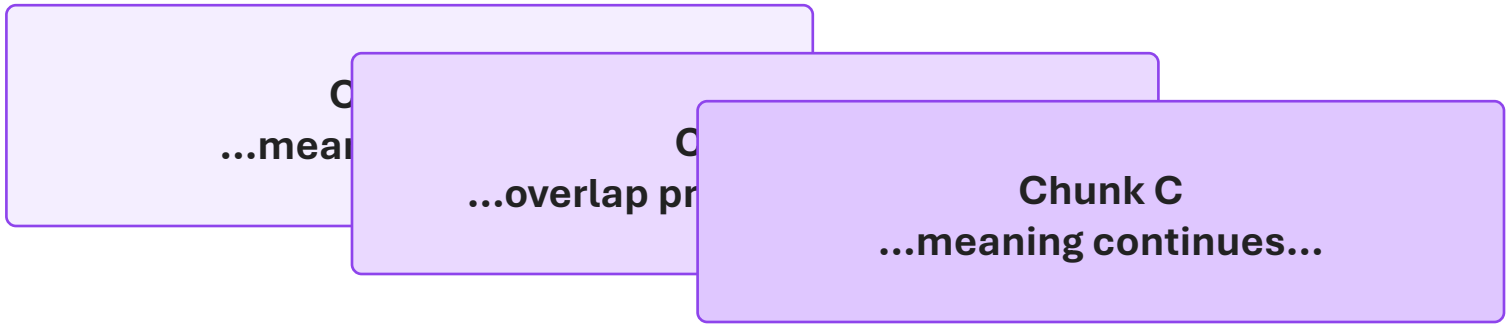
K = 8

Too large
noise + cost

Choosing K is a tradeoff: too small misses facts, too large adds distraction.

Chunking with Overlap

Chunking with Overlap (Production Pattern)



Overlap reduces boundary loss when one idea spans multiple chunks (tradeoff: more tokens).